

See discussions, stats, and author profiles for this publication at: <https://www.researchgate.net/publication/394887593>

Formal query updating to generate problem specific fibration abduction attention algorithms

Preprint · August 2025

DOI: 10.13140/RG.2.2.33029.41446

CITATIONS

0

1 author:



[Gary Nan Tie](#)

Mu Risk LLC

97 PUBLICATIONS 27 CITATIONS

SEE PROFILE

Formal query updating to generate problem specific fibration abduction attention algorithms

Gary Nan Tie, Aug 23, 2025

Abstract We introduce a formalism to construct problem specific query update algorithms founded on standard attention and fibration attention primitives.

Introduction

Given queries $q_i \in Q$, $i \in [n]$ and a key-value pairing $p: K \rightarrow V$, $p(k_j) = v_j$, $j \in [m]$,
We introduce novel ways to refine and update query q_i .

Basic steps:

Step A

Let score be similarity measure, like a reproducing kernel.
Let probability $a_{ij} = \text{softmax score}(q_i, k_j)$ and let $j^* = \arg \max a_{ij}$.
Define $q_i' = \sum_j a_{ij} v_j$. Let $A[q_i] = q_i'$.

Step B

Let $m \times \text{id}_K: Q \times K \rightarrow Q' \times K$ and $\text{id}_{Q'} \times p: Q' \times K \rightarrow Q' \times V$,
 $\text{id}_Q \times p: Q \times K \rightarrow Q \times V$ and $m \times \text{id}_V: Q \times V \rightarrow Q' \times V$.
Suppose the square commutes, that is
 $(\text{id}_{Q'} \times p) \circ (m \times \text{id}_K) = (m \times \text{id}_V) \circ (\text{id}_Q \times p)$.
Learn diagonal lifting $\sigma: Q \times V \rightarrow Q' \times K$ so that both triangles commute,
that is $\sigma \circ (\text{id}_Q \times p) = m \times \text{id}_K$ and $(\text{id}_{Q'} \times p) \circ \sigma = m \times \text{id}_V$.
That is to say $\text{id}_{Q'} \times p$ is fibration, see the references for details.
Define $q_i^* = \text{proj}_1 \sigma(m(q_i), v_{j^*})$. Let $B[q_i] = q_i^*$.

Notation

Let B^*A denote concatenation of Step A followed by Step B with $m(q_i) = q_i'$.

Let $AB[q_i] = A[B[q_i]]$ and $BB[q_i] = B[B[q_i]]$.

Each word with alphabet $\{A, B, *\}$ represents a way to update a query.

For example, LLM FibChat uses $AB...B$ to mitigate hallucinations, while LLM Fibformer uses $B^*A B^*A \dots B^*A$ so that query updates take into account the key-value semantics of pairing p .

This is a formalization of query updating in attention mechanisms. The patterns $`A`$ (standard attention) and $`B`$ (the "fibration step" enforcing semantic consistency via the commutative diagram) are profound primitives.

Examples

Based on this framework, here are several other useful updating patterns, categorized by their intended purpose:

1. Iterative Refinement Patterns

These patterns involve repeatedly applying a set of steps to progressively refine the query, allowing it to interact with the key-value semantics multiple times for deeper understanding.

- * **Pattern:** $A^* (A\text{-Star})$

- * **Composition:** $`A \rightarrow A \rightarrow A \rightarrow \dots`$ (Repeat Step A until convergence or a fixed number of steps).

- * **Operational Meaning:** This is a form of iterative refinement or "multi-hop" attention. The query $`q_i`$ is updated by attending to the context $(`K, V`)$. This new query $`q_{i+1}`$ is then used to attend to the *same* context again. This allows the model to refine its focus based on information gathered in the previous step.

- * **Use Case:** Useful for tasks requiring complex reasoning over a context, like multi-step question answering or refining an initial rough guess. It's a simpler version of the "multi-hop" attention in Memory Networks.

- * **Pattern:** $(B^*A)^* (B^*A\text{-Star})$

- * **Composition:** $`B^*A \rightarrow B^*A \rightarrow B^*A \rightarrow \dots`$ (The Fibformer update applied iteratively).

- * **Operational Meaning:** This is a deep semantic refinement loop. Each cycle first performs a standard attention step ($`A`$) and then immediately grounds and aligns the result ($`B`$) using the fibration constraint. Each iteration forces the query to become more semantically consistent with the key-value pairing $`p`$.

- * **Use Case:** Extremely powerful for tasks where hallucination is a critical failure mode and truthfulness is paramount (e.g., generating medical advice, legal documents, or technical

summaries). It aggressively corrects the query's trajectory at every step.

2. Parallel & Multi-Scale Patterns

These patterns run different update paths in parallel and then combine their results, allowing the model to capture different aspects of information simultaneously.

* **Pattern:** $A + B$

* **Composition:** Compute $A[q_i]$ and $B[q_i]$ in parallel. Then combine them via a learned function (e.g., additive fusion, gating, concatenation + projection): $q_i^* = \text{Gate}(A[q_i], B[q_i])$.

* **Operational Meaning:** This pattern provides a richer update by combining the "freely attended" context from A with the "semantically grounded" context from B . The gating mechanism learns when to trust the standard attention output versus the fibration-corrected output.

* **Use Case:** General-purpose improvement. It can get the benefits of the fibration step's consistency without completely sacrificing the potential serendipitous connections found by standard attention. It's a "best of both worlds" approach.

* **Pattern:** $A \rightarrow (A + B)$

* **Composition:** First compute $q_i' = A[q_i]$. Then compute a residual update using a parallel A and B on this new query: $q_i^* = q_i' + \text{Gate}(A[q_i'], B[q_i'])$.

* **Operational Meaning:** This is a residual block for query updating. The first A provides a base update. The parallel $A+B$ branch acts as a residual adapter, making finer-grained refinements to the already-updated query.

* **Use Case:** Building very deep update networks without suffering from degradation. The residual connection helps with training stability.

3. Conditional Execution Patterns

These patterns introduce a control flow, where the choice of the next update step depends on the current state of the query.

* **Pattern:** $\text{Router}(q_i) \rightarrow \{A, B\}$

* **Composition:** A small, learned "router" network examines the current query q_i and decides whether to apply A or B next.

* **Operational Meaning:** Instead of a fixed sequence, the model learns a policy for applying the primitives. For example, the router might learn: "If the query is confident and well-grounded (q_i is close to many k_j), apply A for refinement. If the query is vague or outlier-like, apply

`B` for strong semantic grounding."

- * **Use Case:** Creating efficient and adaptive models. Computationally expensive steps like `B` (if implemented in a certain way) are only used when necessary, improving efficiency. This is similar to Mixture-of-Experts (MoE) dynamics but for update operations.

- * **Pattern:** While (Uncertain(q_i)) do $B[q_i]$

- * **Composition:** Apply the `B` step repeatedly until a stopping condition is met (e.g., the entropy of the attention distribution α_{ij} falls below a threshold, or the change in q_i becomes small).

- * **Operational Meaning:** This actively seeks a stable, semantically grounded state for the query. It iteratively "corrects" the query until it confidently aligns with a specific key-value pair (or a stable mixture thereof).

- * **Use Case:** "Fact-seeking" behavior. For a query like "What is the capital of France?", the loop would continue until the query unambiguously points to the key "France" and retrieves the value "Paris". This is a very aggressive hallucination mitigation strategy.

4. Pattern inspired by Other Architectures

- * **Pattern:** B (Standalone)

- * **Composition:** Just `B`, using a pre-defined or learned $m(q_i)$ (e.g., $m(q_i) = q_i$ or a simple projection).

- * **Operational Meaning:** This bypasses the softmax attention altogether. The update is *purely* driven by the need to satisfy the commutative diagram constraint for a specific v_j . The query is updated *directly to be consistent with* the retrieved value v_j .

- * **Use Case:** This acts less like attention and more like a fast, deterministic retrieval and fusion operation. It could be used in later layers of a network where the goal is to strongly impose retrieved knowledge onto the representation, similar in spirit to how a K-NN LM might directly bias a model's output.

Conclusion

The power of this formalism is that it allows you to think about attention not as a monolithic function, but as a program built from composable primitives (`A` and `B`). This opens the door for designing new architectures with precise control over the trade-offs between expressive power, semantic consistency, and computational efficiency.

May you design your own specific fibration abduction attention mechanism!

References

[1] 'Fibformer - a fibration transformer'

Gary Nan Tie, Aug 22, 2025

DOI: 10.13140/RG.2.2.36293.10723

Standard attention query updating is incorporated with learning a fibration lifting to take into account key-value pairing semantics.

[2] 'FibChat - It makes stuff up, but with structure.'

Gary Nan Tie · July 2025

DOI: 10.13140/RG.2.2.10549.59364/2

FibChat is a LLM based on fibration attention, that enables coherent structured mitigation of hallucinations.

[3] 'Fibrations explain all you need!

— Fibrations, Abduction, Attention

Gary Nan Tie, Mar 4, 2025

DOI: 10.13140/RG.2.2.35984.52488

To conceptualize and perform abductive machine learning, categorical fibrations are proposed, as they coherently give both context and structure for abductive reasoning via an attention mechanism. Moreover, 2-categorical lifting compatibility conditions ensure consistent hierarchical and parallel explanations.

[4] 'Neural Network Abduction'

Gary Nan Tie, Jun 13, 2025

DOI: 10.13140/RG.2.2.18506.07360/1

Abductive reasoning has a fibration semantics that can be implemented by neural networks; a step towards artificial general intelligence.

[5] 'Parsimonious Neural Network Abduction'

Gary Nan Tie, Jun22, 2025

DOI: 10.13140/RG.2.2.15695.80804

For hypotheses whose effect is manifested by observations, abduction seeks to explain a given observation by finding a hypothesis whose effect is that observation. Abductive reasoning has a fibration semantics that can be implemented by neural networks. In this note we introduce a parsimonious choice of explanation according to one's utility function.

[6] 'Attention Fibration Abduction'

Gary Nan Tie, Jul 2, 2025

DOI: 10.13140/RG.2.2.29060.23680/1

Transformer attention and abductive reasoning are both instances of a weak factorization system fibration, implemented by neural networks.